

# Plan de Branching - Grupo Cordillera

## Estrategia de Branching: Git Flow Adaptado

### 1. Introduccion

Este documento describe la estrategia de branching utilizada en el desarrollo de la Plataforma de Monitoreo Inteligente de Grupo Cordillera. Se adopto un flujo de trabajo basado en Git Flow adaptado a las necesidades del equipo de desarrollo.

### 2. Estructura de Ramas

#### 2.1 Rama Principal: master (o main )

**Proposito:** Contiene el codigo en produccion, listo para ser desplegado.

**Caracteristicas:**

- Solo se reciben merges desde ramas release/
- Cada merge a master representa una version estable
- Se etiqueta con un tag semantico (v1.0.0, v1.1.0, etc.)

#### 2.2 Rama de Integracion: develop

**Proposito:** Rama de integracion donde convergen todas las caracteristicas completadas.

**Caracteristicas:**

- Recibe merges de ramas feature/\*
- Contiene el codigo mas reciente pero aun no probado para produccion
- Base para crear ramas release/

#### 2.3 Ramas de Caracteristicas: feature/\*

**Proposito:** Desarrollo de nuevas caracteristicas de forma aislada.

**Convencion de nombres:** feature/<nombre-descriptivo>

**Ramas creadas en este proyecto:**

| Rama                        | Descripcion                                    |
|-----------------------------|------------------------------------------------|
| feature/patrones-diseno     | Implementacion de patrones Observer y Builder  |
| feature/arquetipos-maven    | Creacion de arquetipos Maven para backend      |
| feature/frontend-components | Componentes frontend y Custom Hooks            |
| feature/documentacion       | Documentacion del proyecto (READMEs, analisis) |
| feature/pruebas-unitarias   | Pruebas unitarias adicionales                  |

**Ciclo de vida:**

1. Crear desde `develop`
2. Desarrollar y hacer commits
3. Actualizar desde `develop` si es necesario
4. Merge a `develop` cuando la característica esta completa
5. Eliminar la rama (opcional)

## 2.4 Ramas de Release: `release/*`

**Proposito:** Preparar una nueva version para produccion.

**Convencion:** `release/<version>`

**En este proyecto:** `release/1.0.0`

**Actividades en esta rama:**

- Pruebas finales
- Correccion de bugs encontrados en las pruebas
- Actualizacion de versiones
- Generacion de changelog

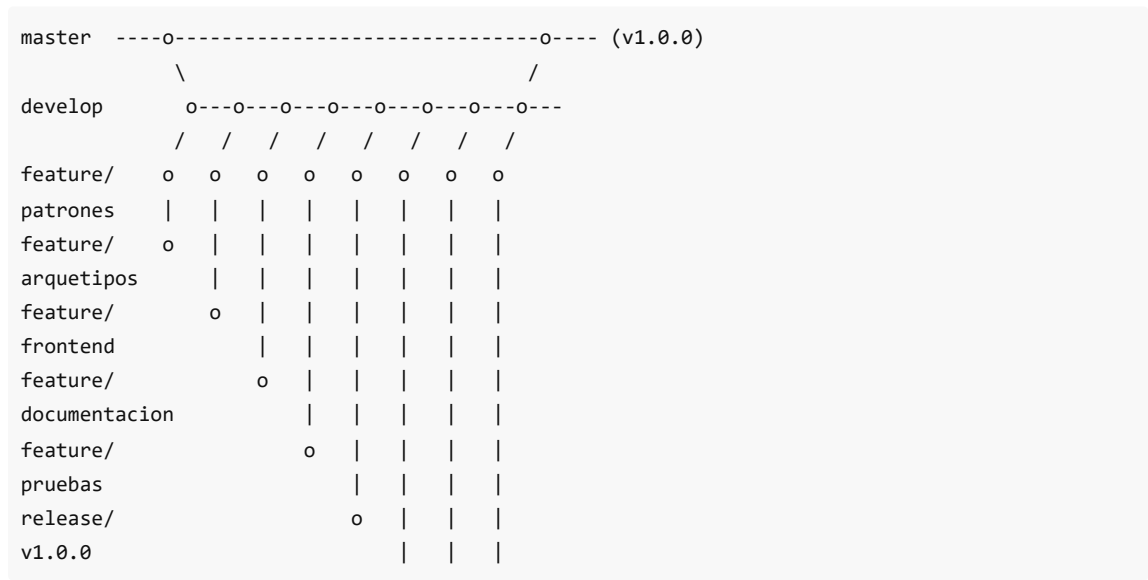
## 2.5 Ramas de Hotfix: `hotfix/*`

**Proposito:** Correcciones urgentes en produccion (no utilizadas en este proyecto).

**Convencion:** `hotfix/<version>`

# 3. Flujo de Trabajo

## 3.1 Diagrama del Flujo



## 3.2 Pasos del Flujo

1. **Desarrollo:** Cada desarrollador trabaja en su rama `feature/*`
2. **Integracion:** Las características completadas se fusionan a `develop`
3. **Preparacion:** Se crea `release/` desde `develop` para pruebas finales
4. **Lanzamiento:** Se fusiona `release/` a `master` y se etiqueta con tag

5. **Sincronizacion:** Se fusiona `release/` tambien a `develop` para incluir correcciones

---

## 4. Comandos Utilizados

### 4.1 Creacion de Ramas

```
# Crear develop
git branch develop

# Crear feature branches
git checkout -b feature/patrones-diseno develop
git checkout -b feature/arquetipos-maven develop
git checkout -b feature/frontend-components develop
git checkout -b feature/documentacion develop
git checkout -b feature/pruebas-unitarias develop

# Crear release
git checkout -b release/1.0.0 develop
```

### 4.2 Merges a Develop

```
# Cada merge usa --no-ff para preservar la historia
git checkout develop
git merge feature/patrones-diseno --no-ff
git merge feature/arquetipos-maven --no-ff
git merge feature/frontend-components --no-ff
git merge feature/documentacion --no-ff
git merge feature/pruebas-unitarias --no-ff
```

### 4.3 Release y Tag

```
git checkout master
git merge release/1.0.0 --no-ff
git tag -a v1.0.0 -m "Version 1.0.0 - Plataforma de Monitoreo"
```

---

## 5. Gestion de Conflictos

### 5.1 Prevencion

- Commits pequeños y frecuentes
- Comunicacion constante entre desarrolladores
- Integracion frecuente con develop (al menos una vez al dia)

### 5.2 Resolucion

En caso de conflictos durante un merge:

1. Identificar los archivos en conflicto: `git status`

2. Revisar las diferencias: `git diff`
  3. Editar los archivos para resolver conflictos manualmente
  4. Marcar como resueltos: `git add <archivo>`
  5. Completar el merge: `git commit`
- 

## 6. Convenciones de Commits

Se utiliza [Conventional Commits](#):

```
<tipo>: <descripcion>
```

### Tipos utilizados:

- `feat` : Nueva característica
- `fix` : Correccion de bug
- `docs` : Cambios en documentacion
- `test` : Adicion o modificacion de pruebas
- `refactor` : Refactorizacion de codigo
- `merge` : Fusion de ramas

### Ejemplos:

```
feat: implement Observer and Builder pattern implementations
test: add unit tests for Builder pattern, event listeners, services, and strategy
docs: add README files for all components
merge: integrate patterns, archetypes, frontend into develop
release: v1.0.0 - Grupo Cordillera monitoring platform
```

---

## 7. Control de Versiones

### Versiones Semanticas (SemVer)

```
vMAJOR.MINOR.PATCH
```

- **MAJOR**: Cambios incompatibles (1.0.0, 2.0.0)
- **MINOR**: Nuevas funcionalidades compatibles (1.1.0, 1.2.0)
- **PATCH**: Correcciones compatibles (1.0.1, 1.0.2)

**Version Actual: v1.0.0**

---

## 8. Estado Actual del Repositorio

### Ramas Existentes

```
master (v1.0.0)
develop
feature/patrones-diseno
feature/arquetipos-maven
feature/frontend-components
feature/documentacion
```

```
feature/pruebas-unitarias  
release/1.0.0
```

## Historial de Commits

```
* release: v1.0.0 - Grupo Cordillera monitoring platform  
* merge: integrate unit tests into develop  
* merge: integrate documentation into develop  
* merge: integrate frontend custom hooks into develop  
* merge: integrate Maven archetypes into develop  
* merge: integrate Observer and Builder patterns into develop  
* feat: implement patterns, archetypes, and project structure  
* feat: initial commit - Grupo Cordillera platform
```

---

## 9. Conclusion

La estrategia de branching basada en Git Flow proporciona una estructura clara y organizada para el desarrollo en equipo. Permite:

- Desarrollo aislado de características
- Integración controlada
- Preparación de releases
- Correcciones rápidas en producción
- Historial claro y rastreable